Lab Report – EE-170L Computer Programming

NAME	ABDUL AHAD
REG.NO	0692
SEMESTER	2nd
LAB	8
DATE	28/4/2026

1. Objectives

- Learn what arrays are and why they are useful.
- Practice declaring arrays in C++.
- Initialize arrays with values.
- Access elements individually using indices.
- Copy arrays using loops.

- Work with character arrays and strings.

2. Theory

An array is a fundamental data structure in C++ that allows programmers to store multiple values of the same type in a single variable. Arrays are stored in contiguous memory locations, which means each element is placed right next to the previous one in memory. This makes accessing elements very fast because the compiler can calculate the memory address of any element using its index.

Arrays are particularly useful when dealing with collections of data, such as storing marks of students, temperatures recorded over a week, or characters in a word. Instead of declaring multiple variables, arrays provide a systematic way to manage data.

Declaration Syntax:

dataType arrayName[arraySize];

Initialization Example: **int**

age[4] = {23, 34, 64, 75};

Array indices start at 0. For example, age[0] refers to the first element, and age[3] refers to the fourth element.

3. Lab Tasks and Programs

Task 1: Array Declaration and Initialization #include

<iostream> // include input-output library using

namespace std; // use standard namespace int main()

{ // main function starts

int numbers[5] = {10, 20, 30, 40, 50}; // declare array with 5 values

for(int i = 0; i < 5; i++) { // loop runs from 0 to 4

cout << numbers[i] << " "; // print each element with space

}

return 0; // end program

}

Task 2: Accessing Array Elements

#include <iostream> // include input-output library

using namespace std; // use standard namespace int

main() { // main function starts

char message[] = "Hello"; // declare character array with word for(int i = 0; message[i] != '\0'; i++) { // loop until end of

```
string
        cout << message[i] << endl;             // print each character on
new line
    }
    return 0;                                   // end program
}
```

Task 3: Copying Arrays

```
#include <iostream>     // include input-output library
using namespace std;  // use standard namespace int
main() {    // main function starts
    int source[5] = {1, 2, 3, 4, 5}; // declare and initialize source
array
    int destination[5];                 // declare destination array
    for(int i = 0; i < 5; i++) {              // loop runs from 0 to 4
        destination[i] = source[i];         // copy each element
    }
    for(int i = 0; i < 5; i++) {                  // loop runs again from 0 to 4
        cout << source[i] << " -> " << destination[i] << endl; // show
both arrays
```

```
    }
    return 0;      // end program
}
```

Task 4: Array Sum

```
#include <iostream>    // include input-output library using
namespace std;    // use standard namespace int main() {   //
main function starts
    int numbers[5];    // declare array of 5 integers int sum = 0;
    // variable to store sum
    cout << "Enter 5 numbers: ";    // ask user for input for(int i =
    0; i < 5; i++) {    // loop runs from 0 to 4 cin >> numbers[i];  //
    take input for each element sum += numbers[i]; // add
    element to sum
    }
cout << "Sum = " << sum;          // display total sum
return 0;                          // end program
}
```

4. Results

- Task 1 displayed the initialized array values: 10, 20, 30, 40, 50.
- Task 2 printed each character of the word “Hello” on separate lines.
- Task 3 showed that the source array was copied correctly into the destination array.
- Task 4 calculated and displayed the sum of user-entered numbers, confirming correct input handling and accumulation.

These results confirm that arrays can store, access, and manipulate data efficiently.

5. Conclusion

In this lab, we explored the concept of arrays in C++. We learned how to declare arrays, initialize them with values, and access elements using indices. We also practiced copying arrays element by element and calculating sums using loops. These exercises demonstrated the importance of arrays in organizing and processing data.

Arrays are not only useful for small programs but also form the basis of more complex applications such as data analysis, simulations, and scientific computing. By mastering arrays, students build a strong foundation for understanding advanced topics like multidimensional arrays, pointers, and dynamic memory allocation. This lab has provided practical experience that will be valuable in future programming tasks and coursework.